

Planning

Instructional Objectives

- The student will get an introduction to planning problems
- They should be able to formulate planning problems in terms of different representation formalisms
- They should be able to cast planning problems as search problems
- Understand why state space search techniques are not appropriate for these problems
- Introduction to search in plan space

Planning

- In the planning problem the task of the agent is to synthesis goal directed behavior
- Select action sequences
 - Handle causal dependencies
- We will restrict ourselves to deterministic and fully observable situations

Why planning is important?

- Intelligent agents must operate in the world
 - Take intelligent actions
 - Compose actions together to achieve complex goals
- Change the world (environment) to suit the needs.
- Agents need to reason about what the world will be like after executing a sequence of actions
 - Need to reason about dynamic environment

A planning problem

- Goal: have a birthday party
- Current situation: agent at home, has flour, does not have butter, does not have sugar
- To do: (sequence of actions)
- Invite friends, buy butter, buy sugar, buy balloons. Decorate house, bake cake

applications

- Mobile robots
- Autonomus agents
 - NASA deep space planning agent
- Simulated environments
 - Goal-directed agents for training or games
- Web and grid environments
 - Composing queries or services

- Scheduling problems with actions, choices as well as resource handling
 - Hubble space Telescope Scheduler
 - Workflow management
- Software test case generation
- Plan based interfaces

Generating plans

- Given
 - A way to describe the world
 - An initial state of the world
 - A goal description
 - A set of possible actions to change the world

Planning problems

- Planning algorithms should take advantage of the logical structure of the problem
- The problem should be expressed in a suitable logical language

Modeling states

- States are modelled in terms of propositions or static variables
 - Complete initial state
 - Partial goal state
- A state is a conjunction of positive literals
 - $\text{Lighted} \wedge \text{hot} \wedge \text{madeofbrass}$
- We can also use first order literals
 - $\text{At}(\text{robert1}, \text{kitchen}) \wedge \text{at}(\text{robert2}, \text{garden})$

Modeling states

- We can also use first order literals
 - $\text{At}(\text{robert1}, \text{kitchen}) \wedge \text{at}(\text{robert2}, \text{garden})$
- Literals must be ground and function free
 - $\text{At}(\text{Father}(\text{fred}), \text{Home})$
 - $\exists x \text{ At}(x, \text{Home})$
- Closed world assumption: any condition not mentioned in a state are assumed false

Representation of goals

- A goal is a partially specified state
 - Ex: $\text{Rich} \wedge \text{Famous} \wedge \text{stays}(\text{Dhaka})$
- A state S satisfies goal g
 - If s contains all the proposition of g
 - $\text{Lighted} \wedge \text{hot} \wedge \text{brass}$
 - Satisfies the goal
 - $\text{Lighted} \wedge \text{hot}$

Modeling planning problem

- States are modelled in terms of propositions
 - Complete initial state
 - Partial goal state
- Actions modeled as state transformation, two frameworks
 - STRIPS
 - ADL

Action representation

- Specification of actions:
- Preconditions that must hold before the action can be executed
- Effect of executing the action
- Action: fly (p, from, to)
 - Precondition:
 - $\text{at}(p, \text{from}) \wedge \text{Canfly}(p) \wedge \text{Airport}(\text{from}) \wedge \text{Airport}(\text{to})$
 - Effect: $\text{At}(p, \text{from}) \wedge \text{At}(p, \text{to})$

- Action schema: can represent a number of different actions
- Precondition: a conjunction of function free positive literals. What must be true in a state before an action can be executed
- Effect: a conjunction of function-free literals. How the state changes when the action is executed

Representing change

- As actions change the world OR we consider possible actions, we want to:
 - Know how an action will alter the world
 - Keep track of the history of world states (have we been here before?)
 - Answer questions about potential world states (what would happen if ..?)

The situation calculus

- Situation calculus (mcCarthy): a formalism to model dynamic worlds within FOL
- Key idea: represent a snapshot of the world, called a “situation” explicitly
- Situations are used to index states of the world
- Actions map situations to situations

Fluents

- Properties that change from situation to situation (called fluents) take an extra situation argument
- Ex: On (B,C, s), Clear (A, s)

Block world example

Frame problem

- I go from home to the store, creating a new situation S' where
 - My son is still at home
 - The store still sells bread
 - My age is still the same
- How can we efficiently represent everything that has not changed?

Successor state axioms

- Normally things stay true from one state to the next
- Unless an action changes them
- $\text{At}(X, \text{do}(A, S)) \text{ iff } A = \text{go}(X)$
- Or $\text{At}(X, S) \text{ and } A \neq \text{go}(Y)$

STRIPS

- Representation for actions
 - Preconditions (list of propositions to be true)
 - Delete list (list of propositions that will become false)
 - Add list (list of propositions that will become true)

Action representation

- Actions must specify all the state variables whose value they change
- No disjunction allowed in effect
- Preconditions and effects are proposition

Example problem

- Initial state:
 - At (home), $\neg$ Have (banana), $\neg$ Have(money)
- Goal: Have (banana), Have (money), At(home)
- Actions

Frame problem

- I go from home to the store, creating a new situation s' , in s' :
 - My son is still at home
 - The store still sells bread
 - My age is still the same
- How can we efficiently represent everything that has not changed?
 - Strips provided a good solution for simple actions

Ramification problem

- I go from home to the store, creating a new situation s' , in s' :
 - I am now in Golebazar
 - The number of people in the store went up by 1
 - The content of my pockets are now in the store
- Do we want to say all that in the action definition?

Solution to the ramification problem

- In strips some facts are inferred within a world state
 - E.g. the number of people in the store
 - ‘primitive’ facts, e.g. `At(home)` persist between states unless changed, ‘inferred’ facts are not carried over and must be referred
 - avoids, making mistakes, perhaps

Planning as a search problem

- Given a representation of the initial state, a set of STRIPS operators, and a goal condition we want to achieve:
 - The planning problem is to determine a sequence of actions that when applied to the initial state yields a state which satisfied the goal

Planning as search

- This can be treated as a search problem
 - The initial state is given
 - The actions are operators mapping a state to a new state
 - The goal is satisfied by any state that satisfies the goal
 - Prove planning problem as a search problem

Problems

- Search tree is generally quite large
- The representation suggests some structure
- Each action only affects a small set of facts
- Actions depend on each other via their preconditions
- Planning algorithms are designed to take advantage of the special nature of the representation

Planning with state space search

- Forward state space search
 - Search from what is known in the initial state and apply operators in the order they are applied
- Backward state space search
 - Search from the description of the goal and identify actions that help to reach the goal

comparison

- Forward search
 - Branching factor can be extremely high
 - All applicable actions considered from each state, includes many irrelevant actions
 - Search is very inefficient without an accurate heuristic
 - Progressive planner

- Backward search
 - Allows us to consider relevant action
 - However, there can still be irrelevant actions
 - Branching factors usually smaller
 - Regressive planner

questions

- Explain how planning systems differ from classical search techniques
- Formulate the blocks world planning problem

Planning -II

Instructional objectives

- The following planning algorithms will be explained
 - Forward search
 - Backward search
 - strips planning
 - Partial order planning

Search in planning

- Planning problem can be cast as search problem
- Given an initial state at a goal state
- How do we generate a sequence of actions?
- Forward state space planning

Forward planner

- Search queue: (Initial state)
- Loop
 - Pick a state from the search queue
 - If it is a goal state, terminate, and return the path from the initial state to the state
 - Otherwise, apply all the applicable actions in the state
 - “progress” the state through the operator application
 - Put each of the state in the search queue
 - End

Progression

- Forward search ($I \longrightarrow G$)
- Progress (state, goals, actions, path)
- if state satisfies goals, then return path
- else $a = \text{choose}(\text{actions})$
- preconditions (a) satisfied in state
- if no such a, then return failure
- else return
- progress (apply (a, state), goals, actions,
- concatenate (path, a)
- First call: Progress (IS, G, Actions, ())

Problems with forward search

- Branching factor is the number of legal actions. Path length is the number of actions required to achieve the goal.
- In any real world situation there are just too many applicable actions
 - Use heuristics
 - Search in the backward direction

Forward Search Example

Backward search

- The goal does not uniquely specify a state, but is a partial description only
- Given a goal, consider only actions those actually be achieved

Backward search

- An action A is applicable in state S in the backward direction if:
 - The effect of A is consistent with S
 - There is at least one effect of A that is part of S
- The state resulting from applying A in the reverse direction (the result of regressing S through A)

- The state resulting from applying A in the reverse direction (the result of regressing S through A)
 - Precondition of A +
 - The variable value assignments of every state variable not in preconditions of A but in S

- Termination criterion:
- The current backward state's partial assignment is consistent with the variable assignment in the initial state

Progression

- Backward search ($I \leftarrow G$)
- Regress (init-state, current-goals, actions, path)
- if init-state satisfies current goals, then return path
- else $a = \text{choose (actions), s.t. some effect of 'a' satisfies one of current goals}$
- if no such 'a', then return failure(unachievable)
- if some effect of 'a' contradicts some of current goals
- then return failure (inconsistent state)
- $CG = \text{current goals} - \text{effects (a)} + \text{preconditions (a)}$
- If current goals $\neq CG$, then return failure (useless)
- Regress (init-state.....

STRIPS Planner

- Divide and conquer
- To create a plan to achieve a conjunction of goals
- Create a plan to achieve one goal and
- Then create a plan to achieve the rest of the goals

STRIPS Planner

- To achieve a list of goals:
 - Choose one of them to achieve
 - If it is not already achieved
 - Choose an action that makes the goal true
 - Achieve the preconditions of the action
 - Carry out the action
 - Achieve the rest of the goals
 - This algorithm is unsound (why?)

STRIPS-Undo achieved goals

- The STRIPS algorithm as above is unsound
 - Achieving one sub-goal may undo already achieved sub-goals

FIXING STRIPS

- Two ideas to make STRIP sound:
 - Protect sub-goals so that once achieved, they can't be undone
 - Protecting sub-goals makes STRIPS incomplete
 - Re-achieve sub-goals that have been undone
 - Re-achieving sub-goals finds longer plans